Project Name: Safety Always Watch (SAW)

Team Members:

1. Vio Gustian Nur Alamsyah
2. Nur Aji Ananta

Problem (The "Why")

- **Current Issue:** PPE violations frequently occur across work areas with varying safety rules. Because supervisors cannot monitor everywhere continuously, many violations go unnoticed or are discovered late.

- **Impact:** Undetected violations increase the risk of workplace accidents, while manual monitoring of multiple areas consumes too much of a supervisor's time and attention.

Proposed Solution

- **Our Approach:** Safety Always Watch (SAW) system uses YOLO.NET to detect people and PPE from cameras. Each area can have different safety rule configurations according to its needs.

  For example:
  Area A: Requires the use of safety goggles and gloves.
  Area B: Requires the use of a helmet and a safety vest.
  Area C: Requires the use of a lab coat, safety goggles, gloves, and shoes.

  When the system detects any non-compliance with the rules of that area, the incident will be processed by a .NET backend and can be forwarded to a dashboard, along with Telegram notifications for the supervisors.

- **Key Value:** Supervisors can obtain a real-time overview of the safety conditions across various areas without the need for continuous manual

monitoring and the system also helps organizations implement more flexible safety rules, as PPE requirements can be configured based on the specific characteristics and risks of each area.

Core Features

- **Feature 1: Real-time Monitoring & PPE Detection**
  Uses YOLO.NET to detect PPE compliance and missing equipment in real-time across customizable Safety and Risk Zones.

- **Feature 2: Area Configuration & Safety Rules**
  Allows supervisors to easily customize mandatory PPE, risk levels, and camera assignments for different work environments.

- **Feature 3: Safety Score & Incident Management**
  Tracks area safety with a 100-point score that drops based on violation severity, logging all incident details and triggering warnings at set thresholds.

- **Feature 4: Dashboard & Notifications**
  Provides a centralized React dashboard showing safety scores, violation stats, and camera statuses, along with automated Telegram alerts for critical violations.

High-Level Architecture & Quality Assurance

- **Architecture Pattern:** Clean Architecture strictly adhering to SOLID principles.
- **Design Patterns Used:** Repository Pattern, Dependency Injection.
- **Code Quality Goal:** Must Have 0 warnings as possible (scanned via static analysis), and 100 % Unit Test Coverage.

Identified Challenges & Risks

- **Challenge 1: Computer Vision Accuracy**
  Environmental factors such as poor lighting, difficult camera angles, occlusions, crowded areas, object distance, and clothing that resembles PPE can reduce accuracy and cause false positives or negatives.

- **Challenge 2: Model & Dataset Limitations**
  Detection quality heavily relies on a comprehensive training dataset that accurately reflects real-world conditions, including various PPE types, lighting, camera angles, and diverse work environments.

- **Risk 1: Alert Fatigue**
  If the system issues warnings too frequently, both users and supervisors may experience alert fatigue. Supervisors can become so accustomed to receiving too many notifications that their level of attention to each warning decreases.